

Faculty of Engineering

Internal Design Document

InfluxDB Design for Distributed Real-Time Telemetry System for QUT Microgrid

Date: October 12, 2026

Student Engineer: Dylan Hessing

Student ID: n11245310

Supervisor: Assoc. Prof. Geoff Walker

Version History

Version	Date	Author	Changes / Comments
1	April 21, 2026	Dylan Hessing	Initial Design
2	October 11, 2026	Dylan Hessing	Updated to as-built implementation

Contents

1 Overview **4**

2 Scope **4**

3 Architecture Overview **5**

4 InfluxDB Version Selection **5**

5 Schema & Data Model **6**

5.1 Organisation 6

5.2 Buckets 6

5.3 Measurement 7

5.4 Tags 7

5.5 Fields 7

5.6 Timestamp 8

5.7 Representative Schema 8

6 Ingestion Pipeline **9**

6.1 Overview 9

6.2 Telegraf Configuration 9

6.3 Burst Payload Path 10

6.4 Write Batching 11

6.5 QoS Alignment 11

6.6 Error Handling 11

7 Full-Rate Waveform Ingestion **11**

7.1 Purpose 11

7.2 Why a Dedicated Service 11

7.3 Schema 12

7.4 Aggregation 13
7.5 Process Structure and Flow Control 13
7.6 Latency 14
7.7 Storage Sizing 14
7.8 Verification 14

8 Query & Retention Policies 15

8.1 Retention Design 15
8.2 Downsampling 15
8.3 Key Query Patterns 16
8.4 Grafana Integration 17

9 PTP Timestamping Integration 18

9.1 Timestamp Authority 18
9.2 Timescale 19
9.3 Clock Domain Consistency 19
9.4 Out-of-Order Writes 19
9.5 PTP Status Logging 19
9.6 Timestamp Quality Validation 20

10 Security 20

10.1 Posture 20
10.2 InfluxDB Access Control 20
10.3 Network Binding 20
10.4 Grafana Authentication 21
10.5 Future Hardening 21

11 Deployment & Infrastructure 21

11.1 Host Configuration 21

11.2 Docker Compose 22

11.3 Persistent Volumes 23

11.4 Environment File 24

12 Validation Approach **24**

12.1 Objectives 24

12.2 Component-Level Tests 24

12.3 End-to-End Validation 25

12.4 Full-Rate Path Validation 25

12.5 Instrumentation 25

13 Resolved Design Questions **26**

1 Overview

This document describes the design of the InfluxDB historian component within the distributed microgrid telemetry system. The historian is responsible for receiving time-series electrical measurements from distributed DAQ nodes, persisting them with nanosecond-precision timestamps derived from PTP-synchronised STM32 firmware, and exposing them for downstream query and analysis.

Version 2 of this document records the historian as built. Where the implementation departed from the original design, the as-built behaviour is described, and design elements that were not implemented are listed as deferred in the relevant section.

2 Scope

This document assumes the MQTT protocol design described in the companion document *MQTT Protocol Design for Distributed Real-Time Telemetry System for QUT Microgrid* (version 2). In particular, it assumes:

- Mosquitto as the broker,
- Telegraf as the historian bridge,
- the four-level topic hierarchy `microgrid/<node_id>/<subsystem>/<metric>`, and
- JSON burst payloads carrying `ts`, `sync`, `dt_ns`, and a 20-element `v` array.

Hardware design, DAQ node fabrication, and RTOS/firmware decisions are out of scope.

3 Architecture Overview

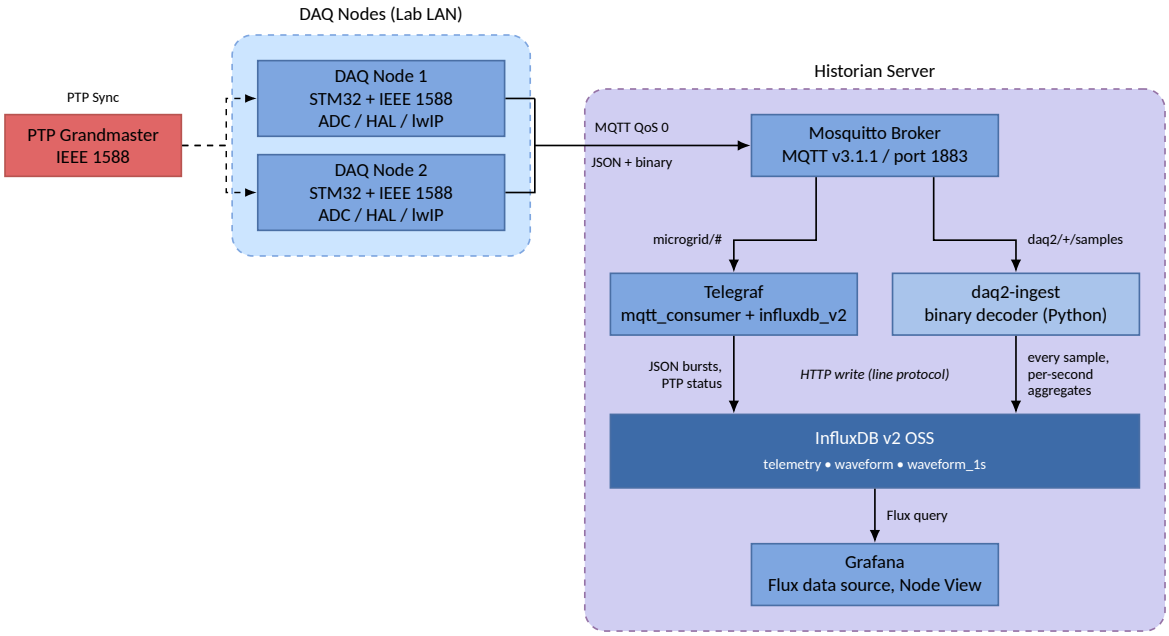


Figure 1: Historian system architecture: PTP-synchronised DAQ nodes publish over MQTT to the Mosquitto broker; Telegraf bridges the JSON topics and `daq2-ingest` the full-rate binary stream to InfluxDB, which Grafana queries via Flux

All components are co-deployed as Docker containers on the same historian server, consistent with the MQTT design's broker configuration. Two subscribers divide the broker's topic space. Telegraf bridges the low-rate JSON topics (`microgrid/#`: bursts and PTP status) to the `telemetry` bucket through configuration alone (Section 6). The full-rate binary stream (`daq2/+/samples`) cannot be handled by Telegraf, so a small custom service, `daq2-ingest`, decodes it and writes every sample to the `waveform` bucket and per-second aggregates to `waveform_1s` (Section 7). The figure's source is `figures/architecture.tex`.

4 InfluxDB Version Selection

InfluxDB v2 OSS is selected as the historian for this project. The justification centres on four properties that are directly required by the system design.

Flux query language and native task scheduling. InfluxDB v2 is the only version for which Flux is the primary, fully–supported query interface, and all dashboard and analysis queries are written in Flux. The original downsampling design relied on Flux scheduled tasks running within the database; as built, downsampling is performed by the ingestion service instead (Section 8.2), but the integrated task scheduler remains available without an external Kapacitor instance, keeping the Docker Compose stack minimal and operationally simple, appropriate for a single–historian research testbed.

Telegraf influxdb_v2 output plugin and HTTP write API. The JSON ingestion path is entirely configuration–driven via Telegraf's mqtt_consumer input and influxdb_v2 output plugin. This plugin (including the

`json_time_key` and `json_time_format = "unix_ns"` features central to this design) is a first-class, mature integration with InfluxDB v2, with extensive documentation and community usage. The full-rate path writes batched, gzip-compressed line protocol directly to the same v2 HTTP API (Section 7).

Token-based access control. InfluxDB v2's token-based authentication model supports bucket-scoped write tokens, which is the basis of the planned security posture (Section 10): write-only tokens scoped to their buckets for Telegraf and `daq2-ingest`, and a separate read-only token for Grafana. This fine-grained token model is native to v2, although the scoped tokens have not yet been issued.

Grafana Flux data source. Grafana's native Flux data source for InfluxDB v2 is stable and well-supported, enabling the dashboard panels (Section 8.4) to query the historian directly in Flux.

Relationship to InfluxDB 3 Core. InfluxDB 3 Core reached general availability in April 2025. However, InfluxDB 3 Core is not positioned as a direct replacement for InfluxDB OSS v2, and v2 is not planned to reach end-of-life. The v3 architecture is a significant departure — it replaces Flux with native SQL and InfluxQL, and integrates a Python processing engine in place of the Kapacitor/task model used in v2. Migrating to v3 would require rewriting all Flux queries documented here and the provisioned dashboard, with no clear benefit at this testbed scale. InfluxDB v2 OSS therefore remains the appropriate and supported choice for a new deployment of this scope.

Docker image pinning note: The latest tag for InfluxDB Docker images will point to InfluxDB 3 Core from late May 2026 onward. The Compose file (Section 11.2) uses the explicit influxdb:2 tag to prevent an unintended upgrade during the thesis validation period. This must not be changed to latest.

5 Schema & Data Model

InfluxDB organises data as:

$$\text{Organisation} \rightarrow \text{Bucket} \rightarrow \text{Measurement} \rightarrow \text{Tags + Fields + Timestamp}$$

5.1 Organisation

A single InfluxDB organisation `qut-microgrid` is used, consistent with the Telegraf output configuration.

5.2 Buckets

Table 1: Buckets

Bucket	Purpose	Retention
`telemetry`	JSON snapshots via Telegraf	90 days
`waveform`	Full-rate samples via `daq2-ingest`	`RAW_RETENTION` (default 7 days)
`waveform_1s`	Per-second aggregates via `daq2-ingest`	Indefinite

The bucket name `telemetry` matches the Telegraf output configuration. The remainder of this section describes the `telemetry` schema; the `waveform` and `waveform_1s` schemas are described in Section 7. The `telemetry_1m` bucket of the original design was superseded by `waveform_1s` (Section 8.2).

5.3 Measurement

Burst points are written to the measurement `mqtt_consumer`, Telegraf's default for the MQTT consumer plugin, and the per-sample points produced from them to `samples`, with the same tags and the fields `v` and `sync`. The electrical quantity is distinguished by the `subsystem` tag rather than by measurement name. This departs from the original design, which mapped each subsystem to its own measurement; a single measurement keeps the Telegraf configuration minimal, and filtering on an indexed tag is equally efficient.

5.4 Tags

Tags are string-valued, indexed, and low-cardinality. All tags are populated by Telegraf from the four-level MQTT topic.

Table 2: Tags

Tag Key	Source	Example Values	Notes
`node_id`	Topic segment 2	`node_01`, `node_02`	Fixed at firmware flash time; MAC/IP addresses explicitly avoided
`subsystem`	Topic segment 3	`voltage`, `current`, `ptp_status`	Quantity; implies units (V or A) for the electrical subsystems
`metric`	Topic segment 4	`phase_a`, `phase_b`, `phase_c`	Phase
`topic`	Full topic	`microgrid/node_01/voltage/phase_a`	Added by Telegraf's `topic_tag`

5.5 Fields

Telegraf's JSON parser flattens the payload's `v` array into indexed fields, so each burst message becomes one point with 22 fields.

Table 3: Fields

Field Key	Type	Meaning
`v_0` … `v_19`	float	Sample values in V or A, from the payload's `v` array
`dt_ns`	float	Interval between samples in nanoseconds
`sync`	float	1 if the node's PTP servo was locked, otherwise 0

PTP status messages (`subsystem` = `ptp_status`, `metric` = `servo`) are stored in the same measurement, one point per Sync, with their own fields:

Table 4: PTP Status Fields

Field Key	Type	Meaning
`offset_ns`	float	Measured clock offset from the grandmaster
`path_delay_ns`	float	Mean path delay used in the offset calculation
`freq_ppb`	float	Frequency correction applied by the servo
`state`	float	Servo state: 0 no Sync yet, 1 estimating frequency, 2 PI loop running
`locked`	float	1 once eight consecutive offsets are within 1 µs
`stepped`	float	1 if this update stepped the clock
`utc_offset_s`	float	TAI−UTC offset applied to `ts`
`utc_announced`	float	1 if that offset came from the grandmaster's Announce, 0 if it is the 37 s default

Each node's firmware version (`subsystem` = `status`, `metric` = `firmware`) is stored as one point per MQTT connection with the string field `firmware`, the output of `git describe` when the firmware was built. It is the only string field kept, through Telegraf's `json_string_fields`.

5.6 Timestamp

The point timestamp is the payload's `ts`: the time of the first sample in the burst, from the node's PTP-disciplined clock, written at nanosecond precision. The historian does not substitute broker receive time or Telegraf processing time. The time of sample k is reconstructed at query time as `_time` $+k\cdot$ `dt_ns`.

5.7 Representative Schema

Table 5: Representative Schema (one point per burst)

_measurement	node_id	subsystem	metric	_field	Example
mqtt_consumer	node_01	voltage	phase_a	v_0	231.45
mqtt_consumer	node_01	voltage	phase_a	…	
mqtt_consumer	node_01	voltage	phase_a	v_19	228.10
mqtt_consumer	node_01	voltage	phase_a	dt_ns	998424
mqtt_consumer	node_01	voltage	phase_a	sync	1
mqtt_consumer	node_01	current	phase_a	v_0	9.82

6 Ingestion Pipeline

6.1 Overview

This section describes the low-rate JSON path (`microgrid/#`), which is fully configuration-driven via Telegraf's MQTT consumer input plugin, regex and Starlark processors, and the InfluxDB v2 output plugin, with no custom code. Telegraf runs as a Docker container on the historian server alongside Mosquitto and InfluxDB. The full-rate binary stream is ingested by a separate service (Section 7).

6.2 Telegraf Configuration

The as-built configuration (`telegraf/telegraf.conf`) is:

```
[agent]
  interval = "10s"
  flush_interval = "1s"
  precision = "1ns"
  omit_hostname = true

[[inputs.mqtt_consumer]]
  servers = ["tcp://mosquitto:1883"]
  topics = ["microgrid/#"]
  qos = 0
  data_format = "json"
  json_time_key = "ts"
  json_time_format = "unix_ns"
  topic_tag = "topic"

[[processors.regex]]
  order = 1
  [[processors.regex.tags]]
    key = "topic"
    pattern = "^microgrid/([^/]+)/([^/]+)/([^/]+)$"
    replacement = "${1}"
    result_key = "node_id"
  # ... likewise ${2} -> subsystem, ${3} -> metric

[[outputs.influxdb_v2]]
  urls = ["http://influxdb:8086"]
  token = "${INFLUXDB_TOKEN}"
  organization = "qut-microgrid"
  bucket = "telemetry"
```

Notes:

- `json_time_key = "ts"` and `json_time_format = "unix_ns"` make Telegraf use the node's PTP-derived timestamp as the InfluxDB point timestamp verbatim, and `precision = "1ns"` pre-

serves its full resolution.

- The full topic is kept as the `topic` tag, and the regex processor extracts `node_id`, `subsystem`, and `metric` from it. This replaced the `topic_parsing` block of the original design.
- Telegraf reaches the broker and database by container name over the internal Docker network.
- The InfluxDB token is injected via an environment variable and is never stored in the configuration file. The broker requires no credentials (see the MQTT design document).

6.3 Burst Payload Path

Each JSON burst `{"ts", "sync", "dt_ns", "v": [20 values]}` is parsed into a single point: Telegraf's JSON parser flattens the `v` array into fields `v_0` to `v_19`, and `sync` and `dt_ns` become numeric fields. The parser cannot reconstruct per-sample timestamps itself. Booleans and strings are dropped by the parser, which is why the firmware sends `sync` as an integer.

Burst Splitting A Starlark processor, run after the regex processor, splits each burst into individually timestamped points. For every burst point it emits 20 additional points in the `samples` measurement, sample k at `ts` $+k\cdot$ `dt_ns`, carrying the burst's tags and the fields `v` and `sync`:

```
[[processors.starlark]]
  order = 2
  source = '''
def apply(metric):
    if "dt_ns" not in metric.fields or "v_0" not in metric.fields:
        return metric
    dt = int(metric.fields["dt_ns"])
    sync = metric.fields.get("sync")
    out = [metric]
    for k in range(1024):
        key = "v_" + str(k)
        if key not in metric.fields:
            break
        sample = Metric("samples")
        for name, value in metric.tags.items():
            sample.tags[name] = value
        sample.fields["v"] = metric.fields[key]
        if sync != None:
            sample.fields["sync"] = sync
        sample.time = metric.time + k * dt
        out.append(sample)
    return out
'''
```

The original burst point is retained alongside the samples, so that existing dashboards and the liveness check are unaffected; at the JSON path's rate the duplication is negligible. This path is the low-rate fallback for when the full-rate stream is not in use. The full-rate binary stream on `daq2/<node_id>/samples` lies outside Telegraf's subscribed topic tree and is ingested separately (Section 7).

6.4 Write Batching

Table 6: Write Batching

Parameter	Value	Notes
`metric_batch_size`	1000 points (default)	Maximum points per write request
`flush_interval`	1s	Maximum time between flushes

At the as-built publish rate (six bursts per node every 5 s, plus one PTP status message per second) the write load is small. The 1 s flush interval keeps PTP status as current as the Node View dashboard, which refreshes every second.

6.5 QoS Alignment

Telemetry is published at QoS 0, consistent with the MQTT design. The ingestion pipeline makes no attempt to detect or fill gaps in the historian record resulting from dropped packets or node disconnection. Gaps are visible in the historian and are acceptable for a research testbed.

6.6 Error Handling

Telegraf drops malformed payloads that fail JSON parsing and logs an error. No dead-letter queue is implemented. This is acceptable given:

- the two-node testbed operates on a wired LAN with low packet loss probability,
- telemetry is QoS 0 by design, so loss is already tolerated at the transport layer, and
- a dead-letter mechanism would add operational complexity disproportionate to the research context.

7 Full-Rate Waveform Ingestion

7.1 Purpose

The JSON path (Sections 5 and 6) stores one 20-sample cycle per channel every 5 s, a small fraction of the approximately 30 kHz the nodes acquire. To store every sample, nodes publish each acquisition packet in binary on `daq2/<node_id>/samples` (see the MQTT design document), and a dedicated service, `daq2-ingest`, writes them to InfluxDB.

7.2 Why a Dedicated Service

Telegraf cannot be used for this stream. Each DAQ2 packet carries 64 frames whose individual sample times must be computed from the header's first-frame time, frame period, and per-channel offsets, which no Telegraf parser supports. At the full rate, two nodes also produce approximately 120 000 points per second,

beyond what a JSON-based pipeline would sustain efficiently. `daq2-ingest` is a small Python service (`daq2-ingest/`) running as a fifth container in the Docker Compose stack. It subscribes to `daq2/+/samples`, decodes each packet, and writes gzip-compressed line protocol to the InfluxDB HTTP API in batches.

7.3 Schema

Table 7: Full-Rate Schema (all points tagged `node_id`)

Bucket	Measurement	Fields	One point per
`waveform`	`voltage`	`a`, `b`, `c` (integer mV)	frame (~33 µs)
	`current`	`a`, `b`, `c` (integer mA)	frame
	`neutral`	`v`, `i` (neutral mode only)	frame
	`daq2_packet`	sequence, frame index, lock and time-valid flags, PTP and configuration epochs, timing uncertainty	packet (64 frames)
`waveform_1s`	`voltage`, `current`, `neutral`	RMS, mean, minimum, and maximum of each field (float V/A), frame count, lock flag	UTC second
	`power`	Real power, apparent power, and power factor per phase and in total, frame count, lock flag	UTC second
	`daq2_ingest`	service statistics	10 s

Design decisions:

- **One point per signal group per frame.** The three phase voltages are sampled simultaneously in one ADC slot, and the three currents in the next, approximately 16.6 µs later. Each group is written as one point at its own exact sampling time, rather than collapsing all six channels onto one timestamp, which would introduce a phase error of 0.3° at 50 Hz between voltage and current.
- **Integer storage.** Raw values are written as integers in mV and mA, which preserves the same 0.001 V/A precision as a three-decimal float. InfluxDB delta-encodes integers, and consecutive samples at 30 kHz differ only slightly, so integers compress approximately 4.8 times better than floats on noisy waveform data (Section 7.7).
- **Only trustworthy timestamps are written.** The firmware clears a packet's time-valid flag whenever the PTP servo adjusted the clock rate during it, which happens on every Sync (about once a second), as well as across clock steps and failed clock reads. A rate adjustment skews sample times by only nanoseconds over one packet, so such packets are kept if their first-frame time lies within 10 µs of the time predicted from the previous packet and the clock epoch is unchanged; they are recorded with `time_valid` = 0 in `daq2_packet`. Packets spanning a clock step or showing a timestamp discontinuity are skipped and counted, as are packets more than a day from the server's clock, which a node publishes before its first PTP step while its clock still counts from boot. Before this rule, one 64-frame packet per second (0.2% of the data) was discarded.
- **Timescale.** The nodes' TAI timestamps are converted to UTC with the offset the node received in the grandmaster's Announce messages, which each packet carries in its flags, as on the JSON path.

Packets without one (before the node's first Announce) use `TAI_UTC_OFFSET_S`, 37 s by default.

7.4 Aggregation

As it ingests each packet, the service accumulates per-channel sums for the current UTC second and emits the RMS, mean, minimum, and maximum once the second completes. It also computes power per phase following the IEEE 1459-2010 definitions: real power $P = \overline{v \cdot i}$, apparent power $S = V_{\text{rms}} I_{\text{rms}}$, and power factor P/S, with totals as the sums over the phases. Each ADC samples a phase's voltage and then its current one scan slot later (16.64 µs, or 0.3° at 50 Hz), which uncorrected would understate real power by about 0.9% at a power factor of 0.5. Each frame's current is therefore interpolated linearly to its voltage's sampling instant, between that frame's and the previous frame's current samples, using the slot offsets in the packet header; across a gap in the frame sequence, the first frame is excluded rather than interpolated. Unit tests confirm the computed power matches theory to within 0.05% at power factors of 1 and 0.5. All of these are written to `waveform_1s`, which is retained indefinitely, so that long-term trends remain available after the raw data expires. A complete second contains approximately 30 048 frames; a lower `frames` count marks a partial second at start-up or after a gap. This replaces the Flux downsampling task of the original design (Section 8.2).

7.5 Process Structure and Flow Control

Python executes one thread at a time per process, and a single-process design was measured to saturate at approximately 2.8 nodes, at which point the broker silently discarded the excess QoS 0 messages for every node. The service therefore runs as one receiver process, which only reads MQTT, and several worker processes (by default one per CPU core, at most eight). The receiver assigns each newly seen node to the worker with the fewest nodes, and all of that node's packets then go to the same worker, which holds its gap-detection, timestamp-continuity, and aggregation state. Each worker decodes, builds line protocol, and posts batches to InfluxDB over four parallel connections, since each post mostly waits on InfluxDB. If InfluxDB falls behind, up to approximately 10 s of raw data is buffered in memory before the oldest raw lines are dropped; aggregates are never dropped. If a worker's queue fills, the receiver drops that node's packets and counts them. Every loss is counted rather than silent: each process logs, and writes to `daq2_ingest` (tagged by worker) every 10 s, the number of packets and frames received, gaps in the nodes' frame indices (data acquired but not delivered), packets skipped for invalid timestamps, queue and buffer drops, and write errors.

The service stops cleanly. On Docker's stop signal, the receiver disconnects from MQTT and passes each worker a stop marker after the packets already queued; each worker processes them, writes the per-second aggregates for the second in progress (marked as partial by their frame count), and posts every pending batch before exiting, within the 10 s Docker allows. The container runs a minimal init process as PID 1, so the signal reaches the service even during start-up. Stopping the service under a two-node full-rate load took 0.4 s, and every packet received until the disconnect was stored.

7.6 Latency

The service measures how long each packet takes to reach it and to be stored. Each packet's newest sample (its last frame's last-sampled column) is the reference, and two times are taken against it: when the packet arrives over MQTT, and when InfluxDB confirms the write that completes it. The first covers the node's processing, the network, and the broker; the second adds the worker queue, write batching, and InfluxDB itself. A sample additionally waits up to one packet duration (64 frames, about 2.1 ms) on the node before its packet is built, which is fixed by design.

Both times compare the node's PTP-derived timestamp with the server's system clock, so the server must be synchronised to the same grandmaster (`ptp4l` and `phc2sys`), which makes the clock error negligible against latencies of milliseconds. Values are not clamped, so a negative latency exposes a server clock that is not synchronised. Every 10 s, each worker adds the median, 99th percentile, and maximum of both to its `daq2_ingest` point, in milliseconds, as `latency_mqtt_p50_ms`, `latency_mqtt_p99_ms`, `latency_mqtt_max_ms`, and the corresponding `latency_stored_*` fields.

7.7 Storage Sizing

Storage was measured by writing 60 s of synthetic three-phase data for one node, with Gaussian noise added (0.3 V and 0.03 A RMS) so that it compresses no better than a real signal:

Table 8: Measured Storage per Value

Format	Bytes per value	Two nodes, per day
Float (V/A, 3 decimals)	7.9	~246 GB
Integer (mV/mA)	1.63	~51 GB

Integer storage is therefore approximately 4.8 times smaller, and is the default. At approximately 51 GB per day for two nodes, the default 7-day raw retention requires approximately 360 GB, and 30 days approximately 1.5 TB; requirements scale linearly with the number of nodes. The `waveform_1s` bucket grows by approximately 1 MB per day. Raw retention should be set from the historian server's available disk, leaving headroom for InfluxDB compactions, and checked against actual usage once real data is flowing.

7.8 Verification

The service was verified on a development machine (16 cores, 16 GB RAM) in three ways:

- **Unit tests** check packet decoding against the firmware's `daq_packet_t` layout, sample time calculation, per-group timestamps, neutral-mode packets, rejection of malformed packets, integer conversion, gap and invalid-time counting, and per-second RMS (a 230 V sine yields 230.0 V).
- **A load test** published synthetic packets for two nodes at the firmware's real rate (30 048 frames/s each) for 60 s. All 56 342 packets were received, and InfluxDB held exactly 1 802 944 voltage and 1 802 944 current points per node, with no drops, gaps, or write errors. The service used approximately 55% of one CPU core and InfluxDB approximately 77%, with writes occupying approximately 42% of the writer's time.

- The per-second aggregates reported 230.0 V RMS and 30 048 frames for each complete second.

On hardware, a node built with `MQTT_PUBLISH_DAQ2` sustained approximately 470 packets/s (6.3 Mbit/s) with no drops at the node, and the service stored all 30 048 frames per second.

Capacity was measured on a home test server (12 threads, 7.7 GB RAM) with one real node plus synthetic nodes:

Table 9: Measured Capacity

Nodes at full rate	Result	CPU used
1	No loss	ingest ~0.5 cores, InfluxDB ~0.5
5	No loss over 2 minutes; queues flat	ingest ~3.2 cores, InfluxDB ~7.8
7	~9% of packets dropped (CPU exhausted)	load average 16 on 12 threads

Capacity is therefore bounded by the server's CPU, at approximately 1.5 cores of InfluxDB and 0.6 cores of ingestion per full-rate node, and by disk, at approximately 25 GB per node per day.

8 Query & Retention Policies

8.1 Retention Design

Retention is configured at the bucket level in InfluxDB v2, and the bucket-level setting is the sole retention mechanism. The `telemetry` bucket's retention is set on first start by `DOCKER_INFLUXDB_INIT_RETENTION`; the `waveform` buckets' retention is set by `daq2-ingest` when it first creates them.

Table 10: Retention

Bucket	Retention	Rationale
`telemetry`	90 days	Covers the thesis validation period
`waveform`	7 days (default)	Set by disk capacity (Section 7)
`waveform_1s`	Indefinite	Long-term trends at small storage cost

At the as-built publish rate of one burst per channel every 5 s, storage growth is small, and 90 days of data from two nodes is well within the capacity of the historian host.

8.2 Downsampling

The original design specified a Flux scheduled task, `downsample_1m`, aggregating the `telemetry` bucket into an indefinitely retained `telemetry_1m` bucket. This was superseded by the full-rate path: `daq2-ingest` computes per-second RMS, mean, minimum, and maximum values from every sample as it ingests them, and writes them to the indefinitely retained `waveform_1s` bucket (Section 7). Computing aggregates in the ingestion service avoids a scheduled task re-reading around 21 million raw points per

minute, and RMS, rather than the mean used in the original design, is the meaningful long-term quantity for AC waveforms, whose mean over a whole number of cycles is approximately zero.

The `telemetry` bucket itself is not downsampled; at one burst per channel every 5 s, 90-day retention does not constrain storage.

8.3 Key Query Patterns

The following Flux examples cover the primary query patterns, written against the as-built schema.

A single channel, one row per burst with `sync`, `dt_ns`, and `v_0` to `v_19` as columns:

```
from(bucket: "telemetry")
  |> range(start: -1h)
  |> filter(fn: (r) => r._measurement == "mqtt_consumer"
                    and r.node_id == "node_01"
                    and r.subsystem == "voltage"
                    and r.metric == "phase_a")
  |> pivot(rowKey: ["_time"], columnKey: ["_field"], valueColumn: "_value")
```

Data written since the burst-splitting processor was added is also available one point per sample in the `samples` measurement (filter on `_measurement == "samples"` and `_field == "v"`). For older data, per-sample timestamps are reconstructed as `_time` $+k\cdot$ `dt_ns` for field `v_`k; the export script `grafana_export.py` performs this expansion when exporting to CSV with `--samples`.

Cross-node comparison for the same channel, using the first sample of each burst:

```
from(bucket: "telemetry")
  |> range(start: -1h)
  |> filter(fn: (r) => r._measurement == "mqtt_consumer"
                    and r.subsystem == "voltage"
                    and r.metric == "phase_a"
                    and r._field == "v_0")
  |> group(columns: ["node_id"])
```

Cross-node comparisons are only meaningful where both nodes report `sync` = 1 for the bursts concerned.

All channels from a single node (debugging):

```
from(bucket: "telemetry")
  |> range(start: -15m)
  |> filter(fn: (r) => r.node_id == "node_01")
```

PTP lock state over a window:

```
from(bucket: "telemetry")
  |> range(start: -1h)
  |> filter(fn: (r) => r._measurement == "mqtt_consumer"
```

```
                    and r._field == "sync")
  |> group(columns: ["node_id"])
```

8.4 Grafana Integration

Grafana connects to InfluxDB v2 via the native Flux data source. Both the data source and the dashboard are provisioned from the repository, so a fresh deployment needs no manual configuration:

- The data source (`grafana/provisioning/datasources/influxdb.yml`) reads its token from `INFLUXDB_TOKEN`, which Docker Compose passes to the Grafana container. Its uid is pinned to the value Grafana originally generated for it, so that existing panels and the export script continue to reference it.
- The *Node View* dashboard (`grafana/provisioning/dashboards/node-view.json`), which shows an operator everything about one node, is loaded into a *Microgrid* folder. It may be edited in Grafana, but must be exported back to the repository for changes to persist.

Table 11: Node View Panels (selected node via a `node_id` variable)

Panel	Source and query pattern
Top-bar selectors	Node, phase (3-phase or A, B, C) and voltage/current measure (RMS or peak). Every phase-specific panel shows only the selected phase, or all three for 3-phase, with a fixed colour per phase
Status	Online if the node's newest point in `telemetry` or `waveform` is less than 30 s old
Average power and power factor	Mean of the `power` fields for the selected phase over the last 10 complete seconds
PTP state	From the newest servo status: *locked* (firmware lock flag), *tracking* (PI loop running with an offset within 100 µs), or *unlocked*
Power	Real and apparent power for the selected phase, per second, on one shared axis so that the gap between them shows the power factor
Voltage and current	Per-phase RMS (root mean square of per-second RMS values) or peak (largest per-second absolute minimum or maximum), at display resolution. When the time range is 5 s or less, the query switches to raw samples from `waveform`, taking the first sample of each millisecond (in storage, by `aggregateWindow`), since Grafana's time axis resolves only 1 ms
PTP row	Offset, path delay and frequency correction from `ptp_status`, one point per Sync, and the firmware's sample-timing uncertainty
Full-rate waveforms	The newest 100 ms that all three phases have reached in `waveform`, among data at least 2 s old (`daq2-ingest`'s writer threads can store a newer batch before an older one), at the full sample rate, plotted against milliseconds so the axis fits the window
Diagnostics (collapsed)	Completeness: frames stored over frames acquired in the minute ending 5 s ago, from the `daq2_packet` frame indices; the newest seconds are excluded because `daq2-ingest`'s writer threads can store batches out of order. Also `daq2_ingest` loss counters summed over all nodes, per-second frame counts, and ingest latency (median and 99th percentile from sample to MQTT arrival and to storage)

9 PTP Timestamping Integration

9.1 Timestamp Authority

The STM32 nodes are the authoritative timestamp source. Each node disciplines its local clock to the PTP grandmaster using hardware timestamping in the Ethernet peripheral. The `ts` field of each burst is the

time of its first sample, computed in firmware from a hardware timer that shares the PTP clock's 100 MHz source, at the centre of that sample's accumulated ADC burst; it is not broker receive time or Telegraf processing time.

Telegraf's `json_time_key` and `json_time_format = "unix_ns"` configuration ensures this timestamp is used as the InfluxDB timestamp verbatim.

9.2 Timescale

PTP time is TAI. The firmware converts it to UTC before publishing by subtracting the TAI−UTC offset (`currentUtcOffset`) advertised in the grandmaster's Announce messages, so a leap second needs no firmware change. Until the first Announce from the node's master arrives, or if the grandmaster marks the offset as not valid, it uses 37 s, the offset since 2017. If the grandmaster announces an arbitrary (ARB) timescale rather than TAI, no offset is subtracted. Before the servo first locks, the node's clock counts from boot; such early bursts carry `sync` = 0.

9.3 Clock Domain Consistency

All nodes share a single PTP clock domain. The historian server's system clock must be on UTC, synchronised to the same grandmaster or to a sufficiently accurate NTP source, so that Flux query time ranges expressed in wall-clock time correspond correctly to stored point timestamps.

9.4 Out-of-Order Writes

MQTT QoS 0 with no ordering guarantees means Telegraf may deliver points to InfluxDB with timestamps older than recently written points. InfluxDB handles out-of-order writes within a shard natively. Default shard duration in InfluxDB v2 is one week; behaviour at shard boundaries should be verified if a node reconnects after an outage longer than one week, though this is unlikely in a lab testbed context.

9.5 PTP Status Logging

Each burst carries a `sync` field, set to 1 when the node's PTP servo was locked at capture. The servo reports lock after eight consecutive offsets within 1 µs of the grandmaster, with a Sync message received within the last 3 s. Bursts are only started on data whose timestamp is valid (no clock step or rate change during capture), and are restarted if a clock step occurs mid-collection.

In addition, on every Sync each node publishes its servo state on `microgrid/<node_id>/ptp_status/servo`: the measured offset, the path delay, the frequency correction, the servo state, and the lock and step flags (see the MQTT design document). Telegraf stores these as one point per Sync in `mqtt_consumer`, with `subsystem` = `ptp_status` and `metric` = `servo`, so the historian holds a continuous record of synchronisation quality for post-hoc assessment of any data window. The original design's HOLDOVER state is not implemented; a node that stops receiving Sync messages reports unlocked after 3 s.

9.6 Timestamp Quality Validation

Before cross-node comparisons are drawn from the historian, `sync` should be confirmed as 1 for all nodes across the relevant time window. Alignment between nodes can be measured independently of the historian by comparing the 1 pulse-per-second outputs of the nodes on an oscilloscope. This validation is a prerequisite for any result that relies on cross-node temporal alignment.

10 Security

10.1 Posture

The historian operates on a university research lab LAN and is not exposed to the public internet. The security design is therefore pragmatic and aligned with the phased approach documented in the companion MQTT design document. This section describes historian-side measures only; broker-side authentication, ACLs, and TLS are covered in the MQTT design document.

10.2 InfluxDB Access Control

InfluxDB v2 uses token-based authentication for all API access. As built, a single admin token, set on first start from `INFLUXDB_TOKEN` in the `.env` file, is used by Telegraf and `daq2-ingest` to write and by Grafana to read. `daq2-ingest` also uses it to create the `waveform` and `waveform_1s` buckets on first start.

Deferred The design specified scoped tokens, so that a compromised token has a limited blast radius:

Table 12: Planned InfluxDB API Tokens

Token	Scope	Used by
Telegraf write token	Write-only to `telemetry` bucket	Telegraf bridge
Ingest write token	Write-only to `waveform` and `waveform_1s`	`daq2-ingest`
Grafana read token	Read-only on all three buckets	Grafana data source
Admin token	Full organisation access	Local admin / ad-hoc Flux queries

With scoped tokens, the buckets would be created in advance rather than by `daq2-ingest`. Tokens are stored as environment variables and injected at runtime: into the Telegraf configuration via `${INFLUXDB_TOKEN}`, and into `daq2-ingest` and the Grafana data source through their container environments. They are never written to configuration files or committed to version control.

10.3 Network Binding

The InfluxDB HTTP API is published only on the host's loopback interface by the Docker port mapping (`127.0.0.1:8086:8086`), so it is not reachable from the lab LAN directly. Telegraf, `daq2-ingest`, and Grafana reach it over the internal Docker network by container name. Administrators access the InfluxDB UI through an SSH tunnel to the historian server.

10.4 Grafana Authentication

Grafana is published on all interfaces (port 3000) so that dashboards can be viewed from the lab LAN. Anonymous access is disabled, and the `admin` password is set on first start from `GRAFANA_ADMIN_PASSWORD`.

Deferred A separate read-only `viewer` account for demonstrations was designed but not provisioned.

10.5 Future Hardening

Consistent with the MQTT design's phased security model, the following measures are deferred:

Table 13: Deferred Security Measures

Measure	Trigger
Scoped Telegraf, ingest and Grafana tokens	Before any wider deployment
Grafana `viewer` account	If demonstration access is needed
TLS on InfluxDB API (`https://`)	If historian is accessed from outside the lab VLAN
Grafana OAuth / SSO	If multi-user access beyond the research team is required
InfluxDB audit logging	If data integrity evidence is required

11 Deployment & Infrastructure

11.1 Host Configuration

All components (Mosquitto, Telegraf, `daq2-ingest`, InfluxDB, and Grafana) are co-deployed as Docker containers on a single historian server running Ubuntu Server 24.04 LTS on the lab LAN. Co-deployment is justified at the two-node testbed scale. At the full rate, CPU is the limiting resource: approximately 1.5 cores of InfluxDB and 0.6 cores of ingestion per node (Section 7), so the host should be sized to the number of nodes.

Table 14: Component Deployment Summary

Component	Image	Host port	Purpose
Mosquitto	`eclipse-mosquitto:2`	`127.0.0.1:1883`[†]	MQTT broker
InfluxDB	`influxdb:2`	`127.0.0.1:8086`	Time-series database
Telegraf	`telegraf:latest`	none	MQTT to InfluxDB bridge (JSON)
`daq2-ingest`	local build	none	MQTT to InfluxDB bridge (full rate)
Grafana	`grafana/grafana:latest`	`0.0.0.0:3000`	Dashboards

[†] Changed to `1883:1883` when real DAQ nodes are connected.

11.2 Docker Compose

All components are managed via a single Docker Compose file. This ensures reproducible deployment, simplifies inter-service networking, and allows the full stack to be brought up or torn down with a single command, appropriate for a research testbed where the environment may need to be rebuilt. The as-built `docker-compose.yml` is:

```
services:
  mosquitto:
    image: eclipse-mosquitto:2
    container_name: mosquitto
    ports:
      - "127.0.0.1:1883:1883"
    volumes:
      - ./mosquitto/config:/mosquitto/config
      - ./mosquitto/data:/mosquitto/data
      - ./mosquitto/log:/mosquitto/log
    restart: unless-stopped

  influxdb:
    image: influxdb:2
    container_name: influxdb
    ports:
      - "127.0.0.1:8086:8086"
    volumes:
      - influxdb-data:/var/lib/influxdb2
      - ./influxdb/init:/docker-entrypoint-initdb.d
    environment:
      - DOCKER_INFLUXDB_INIT_MODE=setup
      - DOCKER_INFLUXDB_INIT_USERNAME=${INFLUXDB_ADMIN_USER}
      - DOCKER_INFLUXDB_INIT_PASSWORD=${INFLUXDB_ADMIN_PASSWORD}
      - DOCKER_INFLUXDB_INIT_ORG=qut-microgrid
      - DOCKER_INFLUXDB_INIT_BUCKET=telemetry
      - DOCKER_INFLUXDB_INIT_RETENTION=90d
      - DOCKER_INFLUXDB_INIT_ADMIN_TOKEN=${INFLUXDB_TOKEN}
    restart: unless-stopped

  telegraf:
    image: telegraf:latest
    container_name: telegraf
    depends_on:
      - mosquitto
      - influxdb
    volumes:
      - ./telegraf/telegraf.conf:/etc/telegraf/telegraf.conf:ro
    environment:
      - INFLUXDB_TOKEN=${INFLUXDB_TOKEN}
    restart: unless-stopped
```

```
  daq2-ingest:
    build: ./daq2-ingest
    container_name: daq2-ingest
    depends_on:
      - mosquitto
      - influxdb
    environment:
      - INFLUXDB_TOKEN=${INFLUXDB_TOKEN}
      # Raw full-rate retention, only applied when the bucket is first
          created
      - RAW_RETENTION=${RAW_RETENTION:-7d}
    restart: unless-stopped

  grafana:
    image: grafana/grafana:latest
    container_name: grafana
    ports:
      - "3000:3000"
    volumes:
      - grafana-data:/var/lib/grafana
      - ./grafana/provisioning:/etc/grafana/provisioning
    environment:
      - GF_SECURITY_ADMIN_PASSWORD=${GRAFANA_ADMIN_PASSWORD}
      # Read by the provisioned data source (grafana/provisioning/
          datasources/influxdb.yml)
      - INFLUXDB_TOKEN=${INFLUXDB_TOKEN}
      - GF_AUTH_ANONYMOUS_ENABLED=false
      - GF_DASHBOARDS_MIN_REFRESH_INTERVAL=1s
    depends_on:
      - influxdb
    restart: unless-stopped

volumes:
  influxdb-data:
  grafana-data:
```

The InfluxDB and Mosquitto ports are bound to loopback by default; Grafana is published on all interfaces for dashboard access from the lab LAN (Section 10). Sensitive values are passed via environment variables sourced from a `.env` file that is excluded from version control.

11.3 Persistent Volumes

InfluxDB and Grafana state are stored in named Docker volumes. Mosquitto data and logs are bind-mounted to host directories for visibility. The Telegraf configuration is bind-mounted read-only, and Grafana's data source and dashboard are provisioned from `grafana/provisioning/`. `daq2-ingest` is stateless: its image is built from the repository, and it creates its buckets on first start.

11.4 Environment File

A `.env` file in the Compose project root holds all secrets:

```
INFLUXDB_ADMIN_USER=admin
INFLUXDB_ADMIN_PASSWORD=<redacted>   # minimum 8 characters
INFLUXDB_TOKEN=<redacted>
GRAFANA_ADMIN_PASSWORD=<redacted>
RAW_RETENTION=7d                     # optional: full-rate retention
```

This file must not be committed to version control. Its values are applied only on the first start of the containers (`RAW_RETENTION` only when the `waveform` bucket is first created); changing them later requires resetting the volumes. Step-by-step setup instructions are given in the repository `README.md`.

12 Validation Approach

12.1 Objectives

The validation approach verifies that the historian correctly receives, stores, and exposes PTP-timestamped measurements from the DAQ nodes. Three properties are of primary interest:

- **Correctness**: data written to InfluxDB matches the values published by the STM32 nodes
- **Timestamp fidelity**: the PTP timestamp from the STM32 is preserved end-to-end with no substitution or modification
- **Cross-node alignment**: points from two synchronised nodes at the same physical instant share consistent timestamps within the expected PTP accuracy bound

12.2 Component-Level Tests

Before end-to-end validation, each pipeline stage is verified in isolation.

Table 15: Component-Level Validation Tests

Test	Method	Pass condition
Telegraf MQTT subscription	Publish a known burst with `mosquitto_pub`; inspect Telegraf logs	Payload parsed, point written to `telemetry` bucket
Timestamp preservation	Publish a burst with a known `ts`; query InfluxDB and compare	Stored `_time` matches `ts` to nanosecond precision
Tag extraction	Publish to a known topic; query tags on the stored point	`node_id`, `subsystem`, and `metric` match the topic segments
Burst flattening	Publish a 20-sample burst; query the stored point	Fields `v_0` to `v_19` hold the published values in order; `sync` and `dt_ns` present

12.3 End-to-End Validation

End-to-end validation uses the two-node testbed with both STM32 nodes publishing live measurements.

Timestamp fidelity test: A known electrical event (e.g. a step load change) is applied. The time of the event as recorded in the historian for each node, reconstructed from `_time` and `dt_ns`, is compared. The difference should be within the expected PTP synchronisation accuracy. A large discrepancy indicates either a timestamp substitution error in the pipeline or a PTP synchronisation failure. Because each node publishes only one 20 ms window every 5 s, the event must fall within a published window on both nodes; the full-rate binary stream removes this constraint.

Data completeness test: Both nodes publish for a fixed duration. The number of points in the historian for each channel is compared against the expected count of one burst per channel every 5 s. Any gap is investigated against Telegraf and Mosquitto logs and the node's UART counters.

PTP lock test: The `sync` field is queried over a sustained period. It should remain 1 throughout normal operation once the servo has locked. The PTP status points give the offset, path delay, and frequency correction behind it, one per Sync.

12.4 Full-Rate Path Validation

The full-rate path (Section 7) is validated separately:

- **Unit tests** (`daq2-ingest/tests/`) check packet decoding against the firmware's packet layout, sample timing, timestamp handling, gap counting, per-second aggregates, and power at known power factors.
- **Load test:** `tests/load.py` publishes synthetic packets for several nodes at the firmware's real rate; the stored point counts must match the packets sent, with no drops, gaps, or write errors.
- **Completeness:** for live nodes, the frames stored in `waveform` are compared with the span of frame indices in `daq2_packet`, which the Node View's completeness indicator does continuously. Any shortfall is traced through the `daq2_ingest` counters (receiver drops, frame gaps, skipped timestamps, buffer drops, write errors) and the node's UART counters.

12.5 Instrumentation

Ingestion health is monitored through Telegraf's container logs (`docker compose logs telegraf`); through `daq2-ingest`'s statistics, logged and written to the `daq2_ingest` measurement every 10 s and summarised in the Node View's Diagnostics row; and, on the node side, the acquisition and MQTT counters printed once per second on each node's debug UART.

Deferred Telegraf's `internal` input plugin, writing pipeline metrics (points received, write errors, flush latency) to a separate `telegraf_internal` bucket, was designed but not enabled.

13 Resolved Design Questions

The following questions were open when this design was first written, and have been resolved by the implementation.

Table 16: Resolved Design Questions

#	Question	Resolution
1	ADC sample rate per channel	Approximately 30 kHz acquisition (30 048 frames/s). Every sample is stored through the full-rate path; the JSON path also publishes one 50 Hz cycle per channel every 5 s, decimated to approximately 1 kHz (20 samples).
2	Single-sample vs burst payload	Burst for JSON. Each burst is stored as one point with fields `v_0` to `v_19`, and split into one point per sample in `samples`. The full-rate path uses 64-frame binary packets.
3	TAI vs UTC in STM32 PTP stack	PTP runs on TAI; the firmware publishes UTC using the TAI−UTC offset from the grandmaster's Announce messages (37 s until one arrives). The historian server's clock must be on UTC.
4	Grafana on same host or separate	Same host. InfluxDB stays bound to loopback; Grafana is published on port 3000.
5	`sync_state` as string or integer field	Replaced by an integer `sync` field (0/1) in each burst, as Telegraf's JSON parser drops strings and booleans.
6	`min`/`max` downsampling required	Yes: `waveform_1s` holds per-second RMS, mean, minimum, and maximum, computed by `daq2-ingest` (Section 8.2).
7	Historian server hardware confirmed	Operating system confirmed: Ubuntu Server 24.04 LTS running the Docker Compose stack. The lab server's CPU, memory, and disk remain to be confirmed; they set the number of full-rate nodes supported (Section 7) and the raw retention.
8	90-day raw retention vs disk capacity	90 days for the JSON `telemetry` bucket, where growth is small. Full-rate raw data (approximately 25 GB per node per day) defaults to 7 days, set from the server's disk; per-second aggregates are kept indefinitely.